第二章
软件可靠性分析

陈仪香

华东师范大学软件学院可信智能团队TrIG

2025年9月18日

TrIG 华东师范大学软件学院可信智能团队
Trustworthy Intelligence Group

1.1 软件

- 软件=程序+文档



著名计算机科学家徐家福



1.2 软件质量

质： 性质，一种事务区别于其他事务的根本属性

质量：产品、工作等的优劣程度。

陆书平，万森，张秋霞编，现代汉语字典，商务印书馆，2014.7

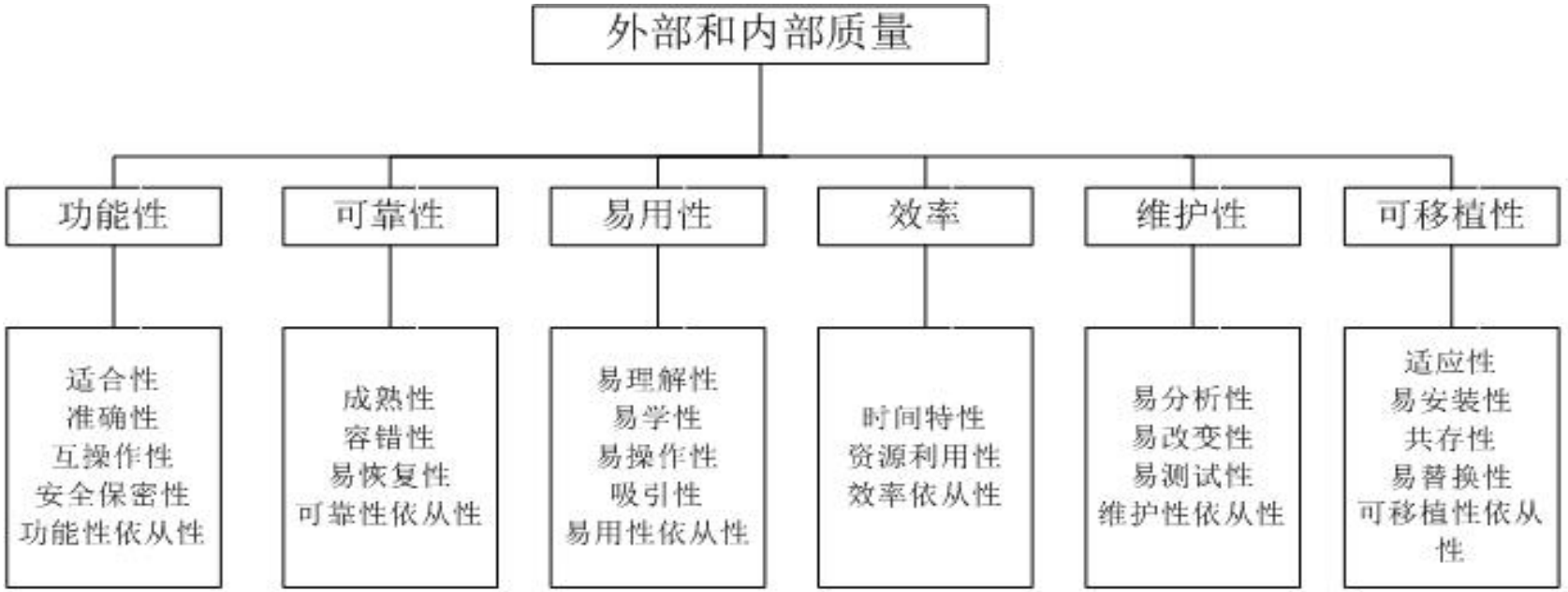
软件质量： 软件的优劣程度

软件=程序+文档（徐家福）

程序=算法+(程序设计语言)+数据结构（Wirth）

1.3 软件质量模型ISO 9126

1991年，ISO/IEC推出ISO/IEC 9126：1991《信息技术 软件产品评价 质量特性及其使用指南》，在此标准中，定义了六种质量特性，并且描述了软件产品评估过程的模型；



ISO 9126: Analysis of Quality Models and Measures，
Chapter · September 2010
DOI: 10.1002/9780470606834.ch10

1.3 软件质量模型

Gilles 1992

Six sessions were carried out with a range of companies from different market sectors: a management consultancy, a software house, a retail organization, a bank, a utility company and a manufacturing company。

The exercise defines relationships as:
(1) If characteristic A is enhanced, then Characteristic B is likely to be degraded, (-) or
(2) If characteristic A is enhanced, then Characteristic B is unlikely to be affected, (0) or
(3) If characteristic A is enhanced, then Characteristic B is likely to be enhanced (+).

1.3 软件质量模型

Table A1. Relationships between criteria: IT consultants

	Characteristic A												
Characteristic B	*FC*	*PC*	*AV*	*E*	*S*	*I*	*U*	*M*	*R*	*A*	*F*	*S*	*R*
Function Correctness (FC)		0	+	−	−	0	0	0	0	0	0	0	+
Performance Correctness (PC)	−		+	+	−	0	+	0	0	−	−	0	+
Added Value (AV)	+	+		+	0	0	+	+	+	+	+	0	+
Efficiency (E)	−	+	0		−	0	−	0	0	−	−		−
Security (S)	0	0	0	0		+	−	0	0	−	−	−	0
Integrity (I)	+	0	0	−	+		0	0	+	−	−	−	+
Usability (U)	+	+	0	−	−	+		0	+	+	+	0	+
Maintainability (M)	0	0	0	−	0	+	0		+	0	0	−	+
Repeatibility (R)	+	0	0	0	0	+	0	0		0	0	−	+
Adaptability (A)	0	−	0	−	−	+	−	0	0		+	−	0
Flexibility (F)	0	−	0	−	−	0	−	0	0	+		+	0
Structuredness (S)	0	0	0	−	0	+	0	0	0	0	0		0
Reliability (R)	+	0	0	−	+	+	0	0	+	−	−	−	

Gilles 关系模型 1992

学习什么内容

第一章 软件质量模型（支撑课程目标1、2）
本章重点解释软件质量的基本概念和模型，以及软件质量发展历史和标准。
讲授软件质量模型的概念和简史。

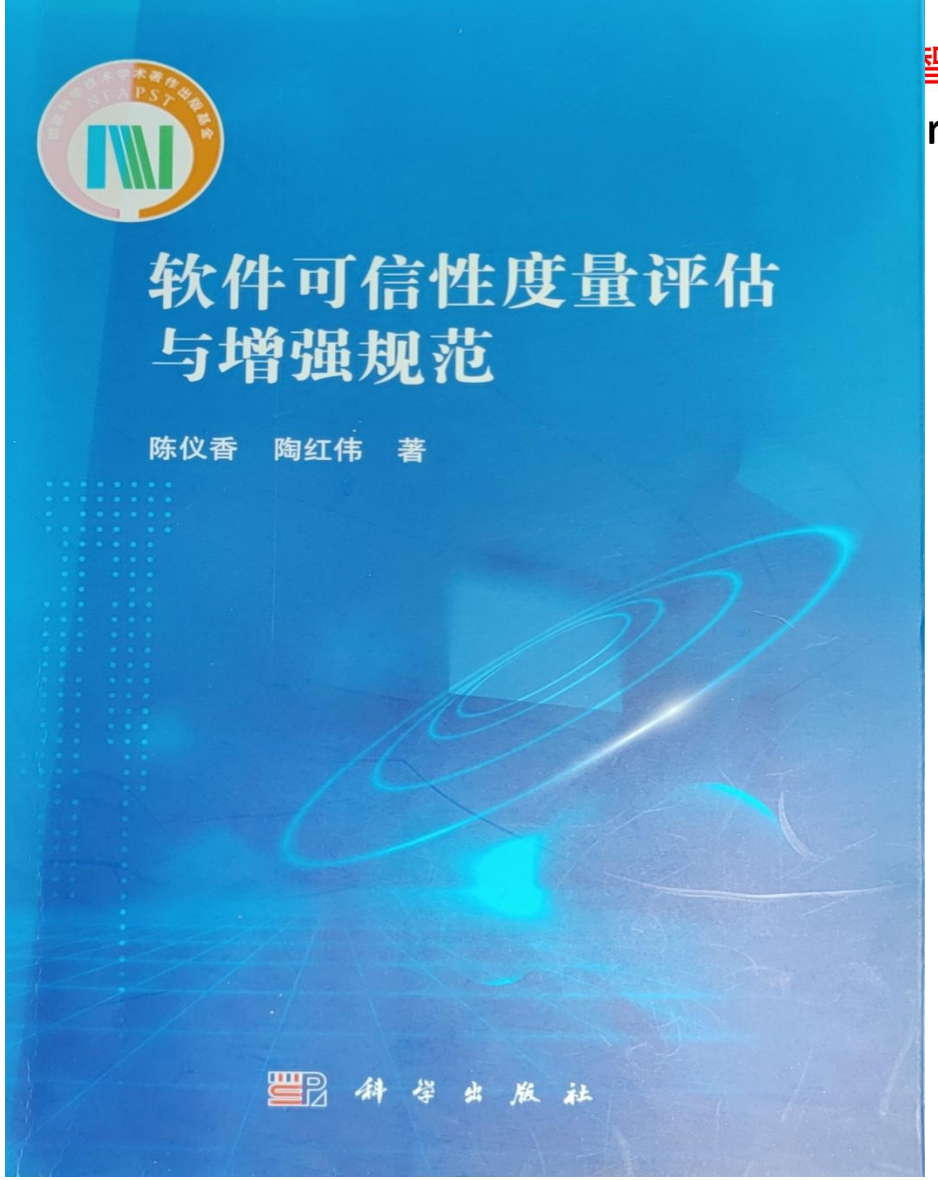
第二章软件可靠性模型（支撑课程目标1、2）
本章讲授软件可靠性概念和度量模型、以及软件可靠性分析。

第三章 软件可信性模型（支撑课程目标1、2）
本章讲授软件可信性的概念和发展过程，软件可信属性模型、软件属性分层模型和软件质量可信度量元模型。

第四章 基于属性的软件可信度量模型（支撑课程目标1、2）
本章讲授基于属性的软件可信度量性质和度量模型、基于属性分解的软件可信度量性质和度量模型、以及分级模型。

推荐教材





2.《软件可靠性工程》，主编：陆民 燕，国防工业出版社，2011.4年

1.《 软件可信性度量评估与增强规范 》（第一版），陈仪香，陶红伟著，科学出版社，2019年。

第二章 软件可靠性分析

2.1 软件可靠性

2.2 软件可靠性分析

2.3 软件可靠性评估

TrIG
华东师范大学软件学院可信智能团队
Trustworthy Intelligence Group

本章参考书



2. 《软件可靠性工程》，主编：陆民 燕，国防工业出版社，2011.4年

第二章 软件可靠性分析

2.1 软件可靠性

2.2 软件可靠性分析

2.3 软件可靠性评估

TrIG
华东师范大学软件学院可信智能团队
Trustworthy Intelligence Group

2.1 软件可靠性

●按照GB/T11457-95-软件工程术语，软件可靠性的定义为：

（1） 在规定的条件下， 在规定的时间内软件不引起系统失效的概率。该概率是系统输入和系统使用的函数，也是软件中存在的缺陷的函数。系统输入将确定是否运到已存在的缺陷（如果有缺陷存在的活）。

（2）在规定的时间周期内所述条件下程序执行所要求的功能的能力。

其中(1)是一个定量的定义，用此定义称之为可靠度更为确切，

而（2）则是一个定性的定义。

2.1 软件可靠性

规定的条件是指：(1) 软件运行的软、硬核环境：软件环境包括运行的操作系统、应用软件、编译系统、数据库系统等；硬件环境包括计算机的CPU、Cache、Memory、I/O等。

(2) 软件操作剖面：不严格地说，是指软件运行的输入空间及其概率分布。

软件的输入空间是指软件所有可能得输入值构成的空间。按照欧空局标准的定义，软件的操作剖面是指”对系统使用条件的定义。即系统的输入值用其按时间的分布或按他们在可能输入范围内得出现概率的分布来定义 “。

2.1 软件可靠性

规定的时间：一般可分为执行时间、日历时间和时钟时间。

- 执行时间是指执行一个程序所用的实际时间或中央处理器时间，或者程序处于执行过程中的一段时间；以秒为单位
- 日历时间指的是编年时间，包括计算机可能未运行的时间。以年为单位
- 时钟时间是指程序执行开始到程序执行完毕所经过的钟表时间，该时间包括了其他程序运行的时间。以小时和分为单位

大多数的软件可靠性模型是针对执行时间建立的，因为真正激励软件发生失效的是CPU时间。

2.1 软件可靠性

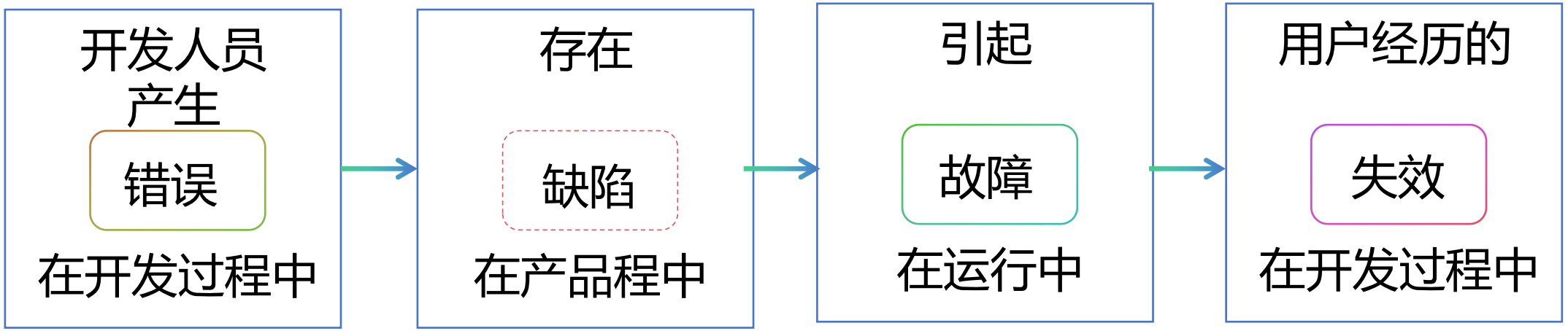
规定的功能：软件的可靠性还与规定的功能密切相关。

所谓规定的功能是指软件应完成的工作。

事先必须明确规定的功能，只有这样，才能对软件是否发生失效有明确的判断。

2.1 软件可靠性

软件失效机理：当软件发生失效是， 我们说软件不可靠，发生的失效数越多，或者发生失效的时间间隔越短，我们认为软件越不可靠。那么软件为什么会失效呢？ 软件失效机理见下图：

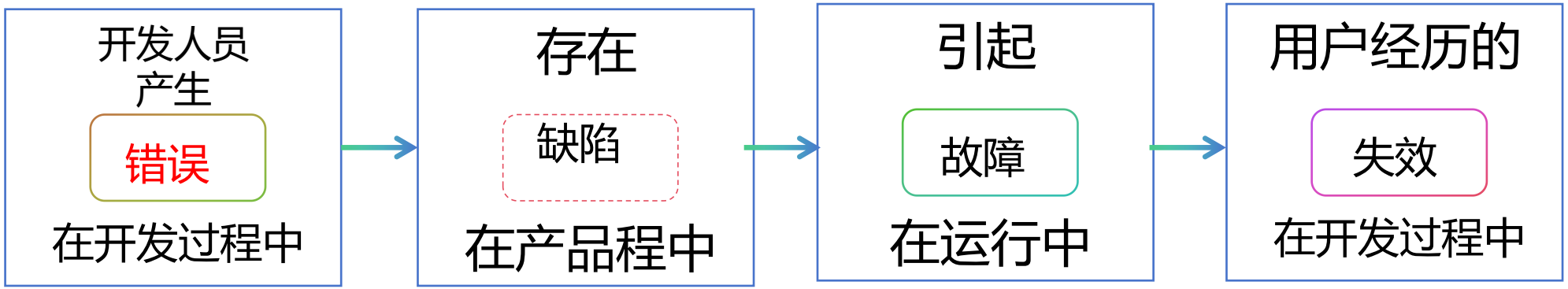


$$y = \frac{1}{x}, \quad z = \sqrt{x + y}$$

2.1 软件可靠性

错误(error)： 指开发人员在开发过程中出现的失误、疏忽和错误。例如，遗漏或误解软件需求规格说明中的用户需求，不正确的翻译或遗漏设计规格说明的需求。

如爱国者导弹的舍二进制的25位以下的数字、火星气候轨道探测器MCO的单位不一致、阿丽亚娜5火箭高度和速度组件SRI的数值位数表示不一致。

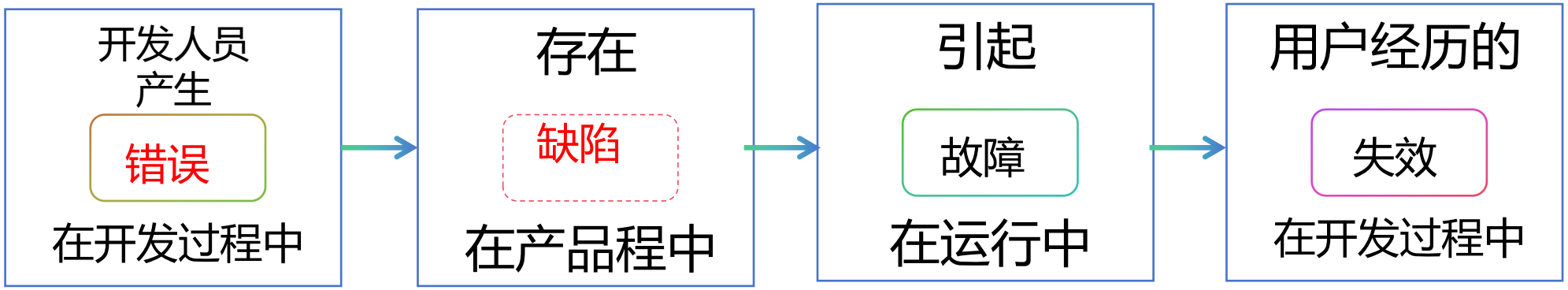


软件失效机理图

2.1 软件可靠性

缺陷(defect)： 指代码中能引起一个或多个失效的错误的编码（步骤、过程、数据定义等），软件缺陷是程序故有的，只要不修改程序去除已有的缺陷，缺陷就会永远留在程序中。

另外，软件文档中不正确的描述也称为缺陷，如不正确的功能需求、遗漏的性能需求等。

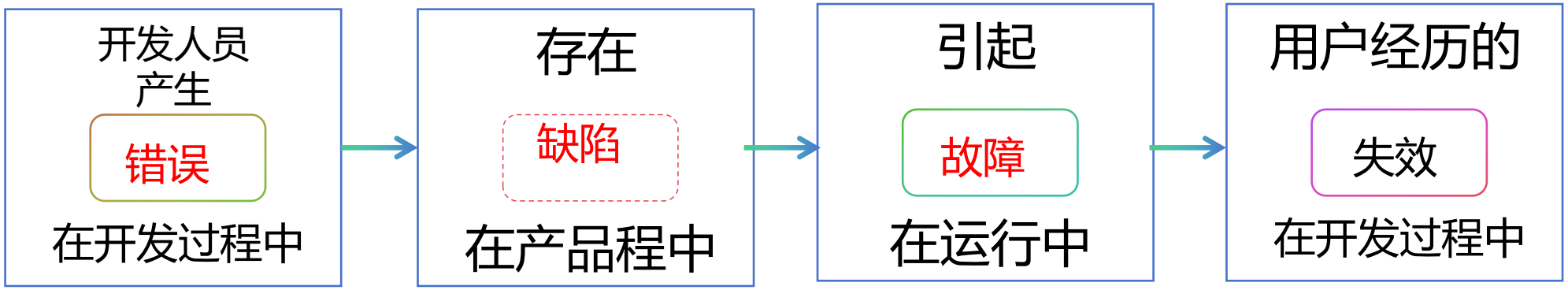


软件失效机理图

2.1 软件可靠性

故障(fault)： 指软件在运行过程中出现的一种不希望或不可接受的内部状态，通常是由于软件缺陷在运行时引起并产生的错误状态。

如不正确的数值，数据在传输过程中产生的偏差。



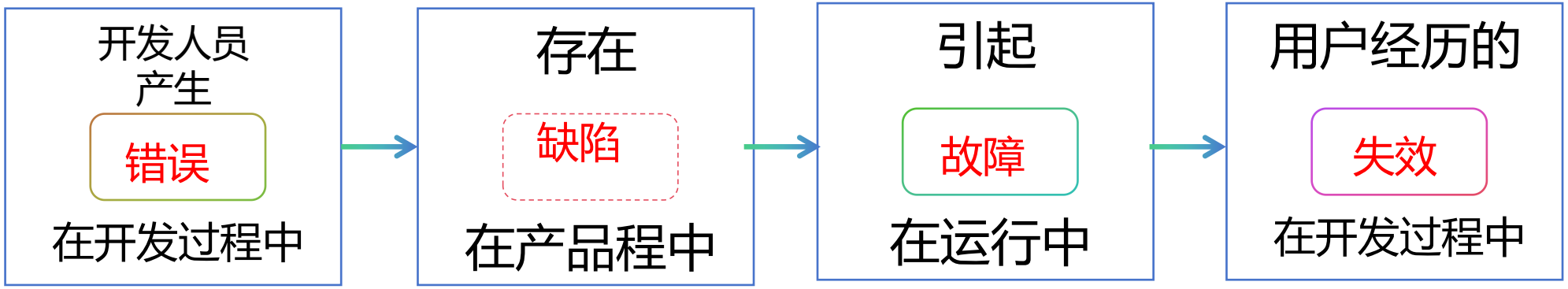
软件失效机理图

2.1 软件可靠性

失效(failure)： 指程序的运行偏离了需求，是动态运行的结果。软件执行遇到软件中的缺陷时可能会导致软件的失效。

如死机、错误的输出结果、没有在规定的时间内响应都是失效。

爱国者导弹的0.000000095的误差夺走28条生命、阿丽亚娜5火箭高度和速度组件SRI的数值错误表示导致火箭发射失败。



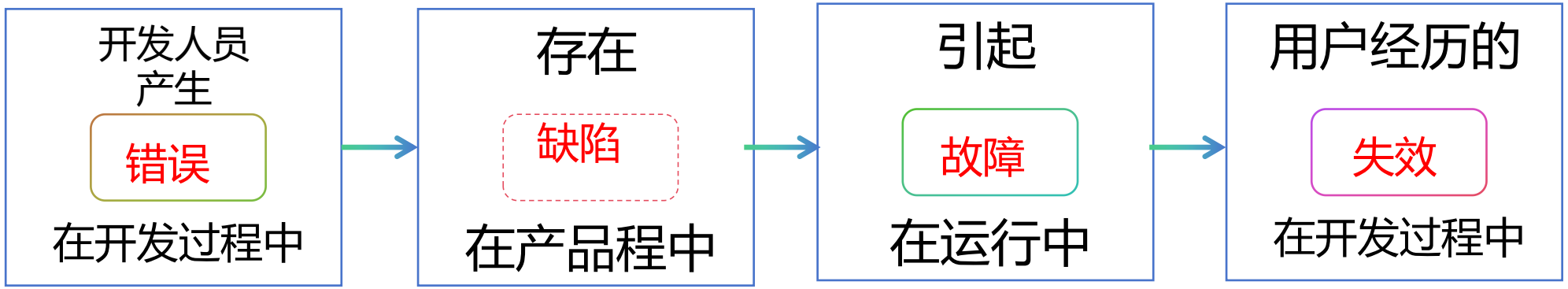
软件失效机理图

2.1 软件可靠性

软件失效的随机性：从软件可靠性的定义可以看出，软件可靠性是使用概率度量的， 因此软件失效的发生是随机的。

但是经验又告诉我们，在使用一个程序是，在其他条件（如初始值、配置）不变的前提下，相同的输入数据会得到相同的输出结果，无论结果是否正确都是确定的，不会存在随机性问题。

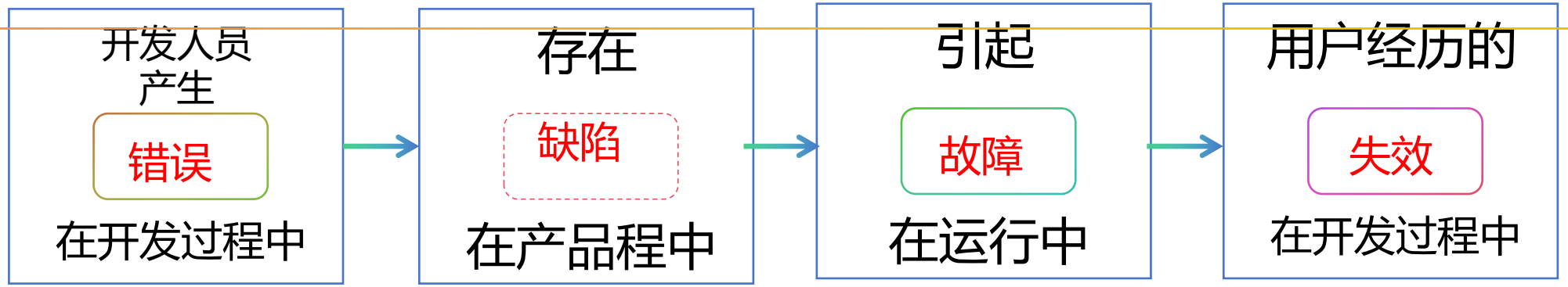
那么如何理解软件失效的随机性问题呢?



2.1 软件可靠性

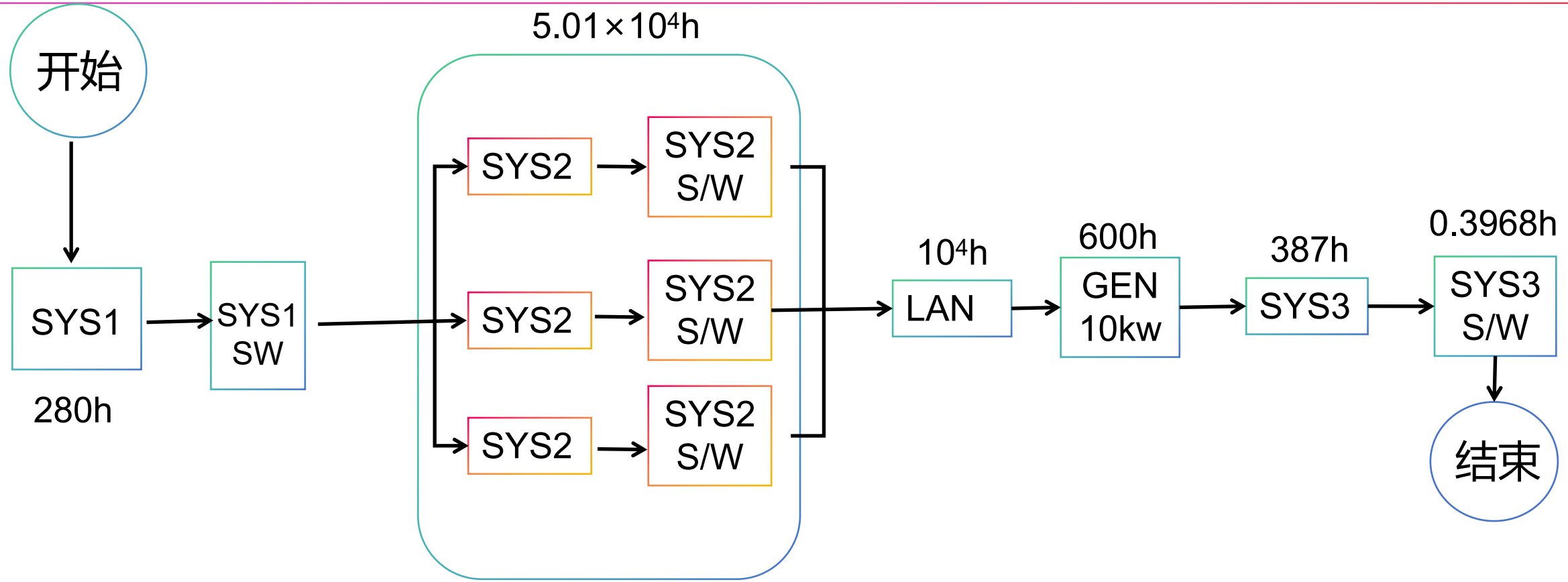
软件失效的随机性：从软件可靠性的定义可以看出，软件可靠性是使用概率度量的， 因此软件失效的发生是随机的。但是经验有告诉我们，在使用一个程序是，在其他条件（如初始值、配置）不变的前提下，相同的输入数据会得到相同的输出结果，无论结果是否正确都是确定的，不会存在随机性问题。 那么如何理解软件失效的随机性问题呢?

首先，程序执行的条件一般是不可预测的、是变化的，因而程序的使用千差万别。此次，缺陷或错误在程序中的位置是未知的。因为由于开发人员造成差错而引起的错误是复杂而难以预计的，即你无法预测程序中哪里存在缺陷。如果你事先知道哪里存在缺陷，你就可以事先把它除掉了。因此，在实际使用软件中，**何时遇到程序中的缺陷从而导致软件失效就呈现出随机性**。



2.1 软件可靠性

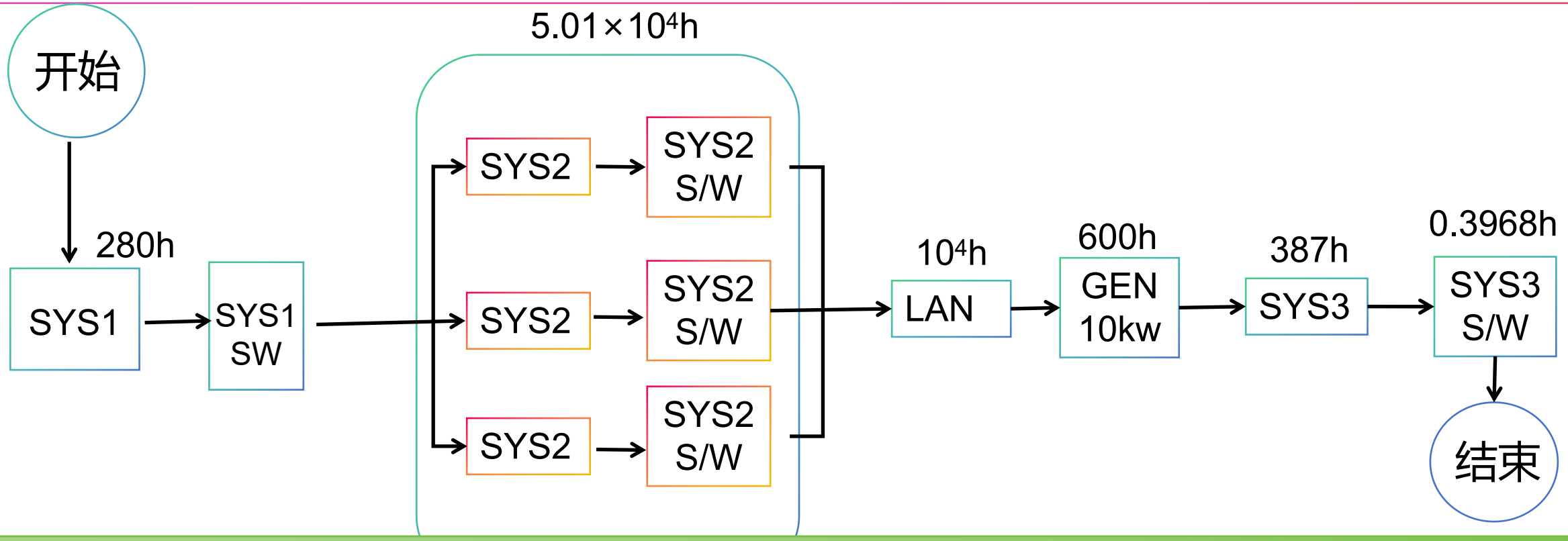
例子：某军用分布式处理系统结构示意图。SYS1、SYS2、SYS3三个分系统以及一个局域网LAN和10kW发电机组成， 而三个SYS2则构成一个冗余系统（至少两个工作即可）



其中SYS1、SYS2、SYS3均包含对应的硬件系统（图中用SYS1、SYS2、SYS3表示）和软件系统（图中使用SYS1 S/W、SYS2 S/W、SYS3 S/W表示）

2.1 软件可靠性

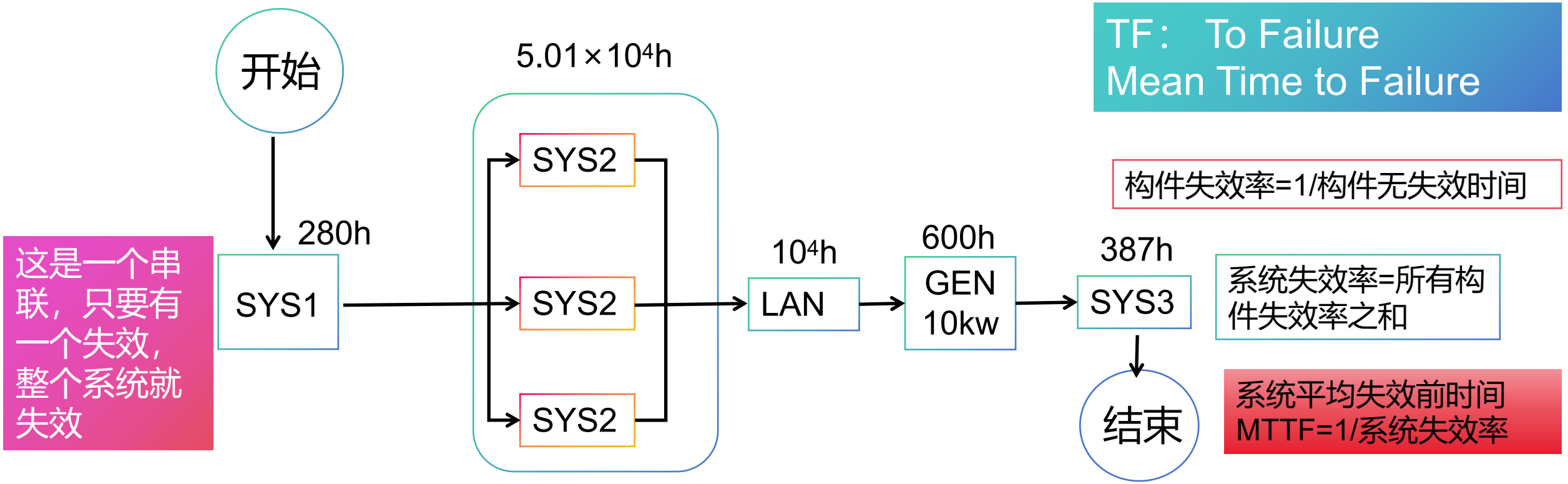
例子：某分布式处理系统结构示意图。SYS1、SYS2、SYS3三个分系统以及一个局域网LAN和10kW发电机组成， 而三个SYS2则构成一个冗余系统（至少两个工作即可）



依据系统可靠性要求，对硬件模块的平均失效时间MTTF(Mean Time To Failure)已经分别预计出并已标记在对应模块的上方，问此设计能否满足MTTF为100h的系统可靠性要求?

2.1 软件可靠性

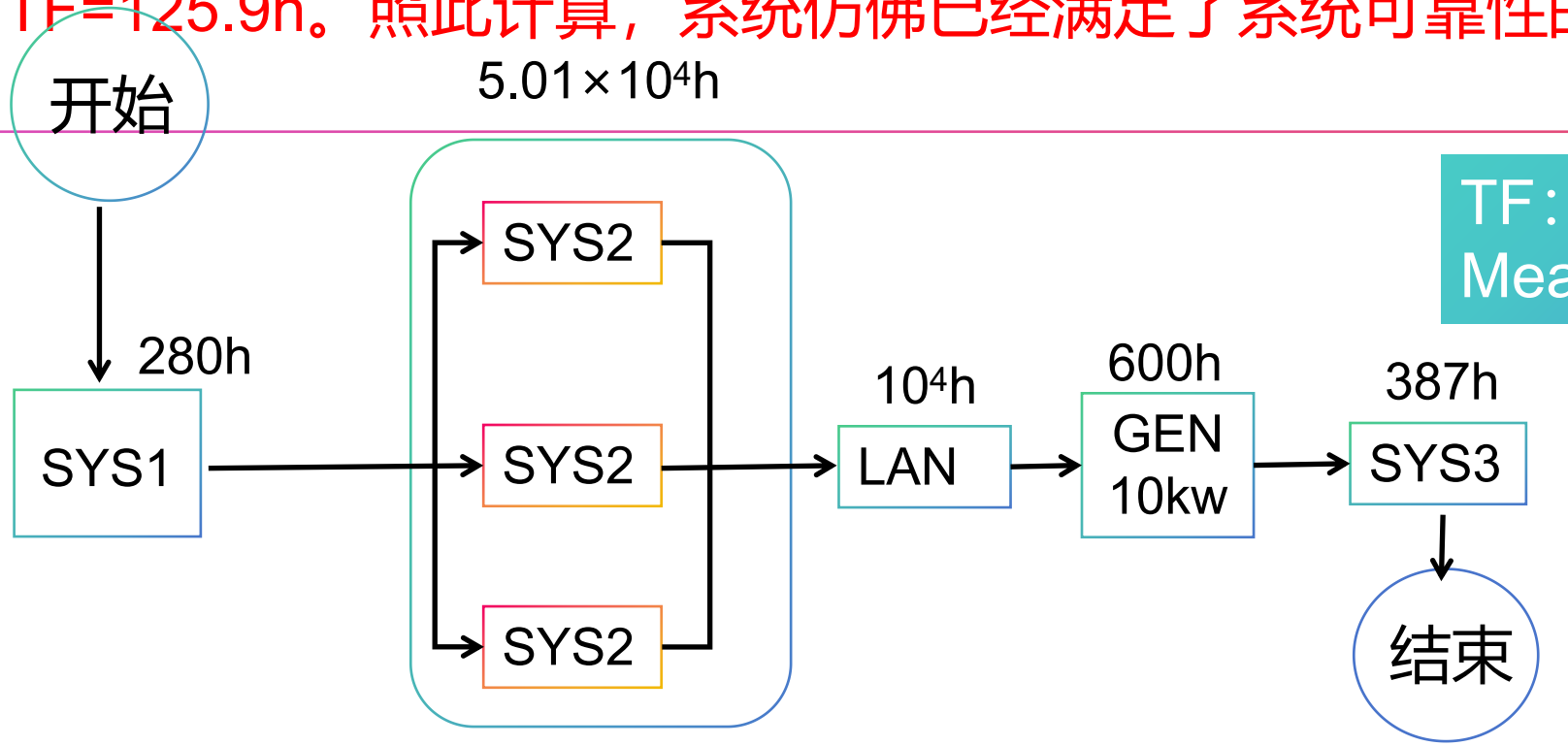
我们分两种情况进行分析：情况1：不考虑软件失效情况。此时整个系统是由SYS1、3个SYS2构成的冗余系统、LAN、GEN以及SYS3构成的一个串联系统其MTTF计算公式:



$$\frac{1}{\mathrm{TFs}}=\sum_{i=1}^{n}\frac{1}{\mathrm{T}_{\mathrm{TF}i}}=\frac{1}{280}+\frac{1}{5.01\times10^{4}}+\frac{1}{10^{4}}+\frac{1}{600}+\frac{1}{387}=0.007942(1/\mathrm{h})$$

2.1 软件可靠性

我们分两种情况进行分析：情况1：不考虑软件失效情况。此时整个系统是由SYS1、3个SYS2构成的冗余系统、LAN、GEN以及SYS3构成的一个串联系统其MTTF计算公式:
系统的MTTF=125.9h。照此计算，系统仿佛已经满足了系统可靠性的初步要求



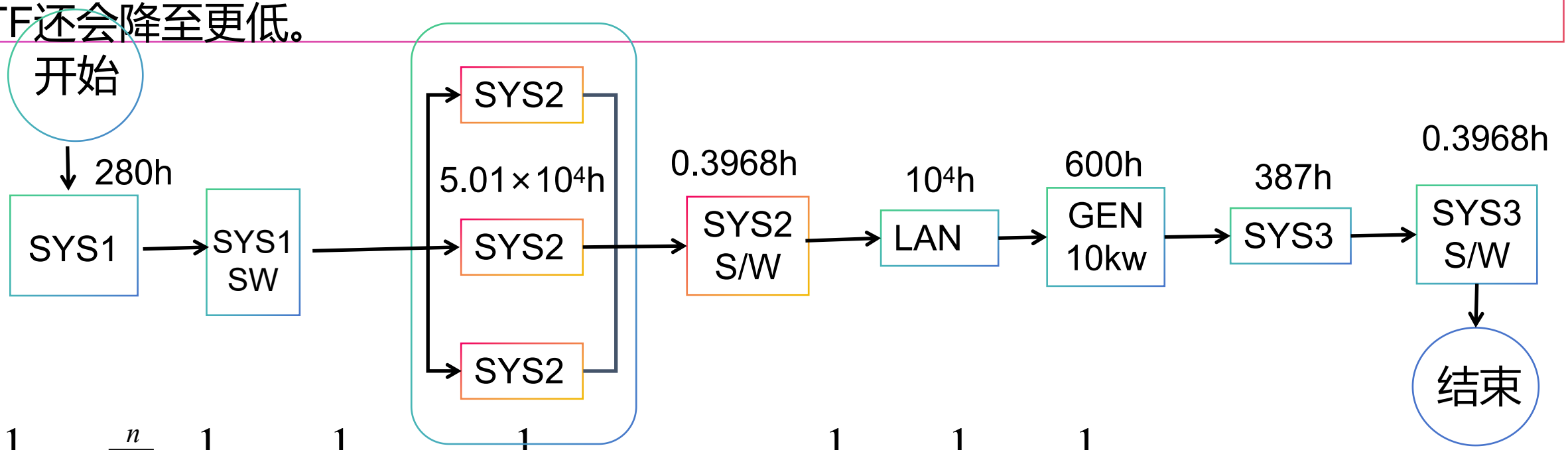
TF： To Failure
Mean Time to Failure

系统平均失效前时间
MTTF=1/系统失效率

$$\frac{1}{\mathrm{TFs}}=\sum_{i=1}^{n}\frac{1}{\mathrm{T}_{\mathrm{TF}i}}=\frac{1}{280}+\frac{1}{5.01\times10^{4}}+\frac{1}{10^{4}}+\frac{1}{600}+\frac{1}{387}=0.007942(1/\mathrm{h})$$

2.1 软件可靠性

情况2：考虑软件失效情况。这个系统分别具有30000行源代码的SYS2和SYS3的初始失效率为2.52/h，即SYS2 S/W和SYS3 S/W的MTTF均为0.396825h。而在SYS2中，因为软件版本是相同的，冗余系统对软件失效无法起到冗余作用，故而考虑软件失效后只是相当于在原系统上同时串联上了SYS2 S/W和SYS3 S/W，可靠性框图如下：（不考虑SYS1 S/W失效情况下），系统MTTF为11.9min，若考虑SYS1软件失效性，则系统MTTF还会降至更低。



$$\frac{1}{TFs}=\sum_{i=1}^{n}\frac{1}{T_{TFi}}=\frac{1}{280}+\frac{1}{5.01\times10^{4}}+2.52+\frac{1}{10^{4}}+\frac{1}{600}+\frac{1}{387}+2.52=5.046(1/h)$$

一个小时能失效五次

2.1 软件可靠性

这个例子告诉我们，软件可靠性对系统可靠性具有重要的影响，要保证系统的可靠性，不仅要对硬件可靠性提出定量要求，同时也必须对软件的可靠性提出定量的要求。

此外，确定软件可靠性要求可以使用户和开发人员找出影响系统整体可靠性的关键部分，对可能不可靠的部分进行重新设计，提高可靠性。

我们将面向软件用户的可靠性度量参数称为软件可靠性参数。

2.1 软件可靠性--度量参数

- 平均失效前时间MTTF和平均失效间隔时间MTBF

1)平均失效前时间MTTF(Mean Time To Failure)

- MTTF是指当前时间到下一次失效时间的均值。

假设当前时间到下一次失效的时间为ξ，ξ具有累计概率密度函数 $F(t)=P(\xi \le t)$，即可靠度函数 $R(t)=1-F(t)=P(\xi > t)$

则基于MTTF的软件可靠度T_{TF}为：

$$T_{TF}=\int_0^{\infty} R(t)dt$$

2.1 软件可靠性--度量参数

- 平均失效前时间MTTF和平均失效间隔时间MTBF

2)平均失效间隔时间MTBF(Mean Time Between Failure)定义

■ MTBF是指两次相邻失效时间间隔的均值。其计算公式

$$\mathrm{MTBF} = \frac{1}{N(t)}$$

■其中t是系统运行时间，N(t)是系统在时刻[0,t]时间段内的故障次数（不能为零）。

■基于MTBF的软件可靠度计算：R(t)=exp(-(t/MTBF))

■如一个系统运行1000小时，出现故障2次， 则MTBF=500,计算在100小时的可靠度：R(100)=exp(-100/500)=exp(-0.20)=0.819

2.1 软件可靠性--度量参数

- 平均失效前时间MTTF和平均失效间隔时间MTBF

2)平均失效间隔时间MTBF(Mean Time Between Failure)定义

- 基于MTBF的软件可靠度计算：R(t)=exp(-(t/MTBF))
- 如一个系统运行1000小时，出现故障2次， 则MTBF=500,计算在100小时的可靠度：
  R(100)=exp(-100/500)=exp(-0.20)=0.819
- 可靠度一般是说几个9，即一个9（0.9），二个9（0.99）， 0.999
- 0.9对应-0.1，1000个小时出现故障1次
- 0.99对应-0.01,10000个小时出现故障1次
- 0.999对应-0.001，100000个小时出现故障1次

这是计算100个小时内可靠度

2.1 软件可靠性--度量参数

- 使用math库计算e的x次方

```
import math

def exp_math(x):
    """使用math库计算e的x次方"""
    return math.exp(x)

# 测试
test_values = [-0.2, -0.01, -0.1, -0.001, 2, 5]
print("使用math.exp()计算:")
for x in test_values:
    print(f"e^{x} = {exp_math(x)}")
```

这是计算100个小时内可靠度

2.1 软件可靠性--度量参数

缺陷密度（Defect Density,DD）： 是软件缺陷的基本度量，可用于设定产品质量目标，支持软件可靠性模型（如Rayleigh模型）预测潜藏的软件缺陷，因而对软件质量进行分析、跟踪和管理。

- 支持基于软件缺陷计数的软件可靠性增长模型(如Musa-Okumoto模型），对软件质量目标进行跟踪并判定能否结束软件测试。
- 用途：这个度量分析每个版本或模块中每千行代码中的缺陷数量。这样可以在一系列版本或模块中追踪软件质量。

适用阶段：适用于需求、设计、编码、测试、使用及维护阶段。

2.1 软件可靠性--度量参数

缺陷密度DD：

数据元素：D是每个版本或每个模块规定严重性等级下的缺陷（或偏差报告）数。

KSLOC是在每个版本或每个模块中可执行代码和非可执行数据声明的千行源代码数(=代码行数/1000)。(Line of Codes, LOC)

缺陷密度DD:

$$DD = \frac{D}{KSLOC}$$

2.1 软件可靠性--度量参数

故障密度FD（Fault Density）：

$$FD = \frac{F}{KSLOC}$$

用途：使用这个度量，可以通过按严重性分类将计算的故障密度与目标值比较来确定是否已经完成足够的测试。

使用阶段：测试、使用和维护

数据元素：F是指每个版本或模块中发现的导致具有规定的严重性等级的失效的唯一（注：唯一是指相同的故障计为一次）故障数。

KSLOC 是在每个版本或每个模块中可执行代码和非可执行数据声明的千行源代码数（1万行代码对应KSLOC为10）。

2.1 软件可靠性--度量参数

需求依从性（Requirements Compliance）：

用途：该度量反映软件需求分析工作的度量。

根据该度量可以确定在需求分析阶段，软件需求规格说明中的

① 需求不一致的比例，

② 需求不完整的比例，

③ 需求曲解的比例，

可以确定主要的需求问题类型，从而有效改进软件需求分析的质量。

适用阶段：需求阶段

2.1 软件可靠性--度量参数

需求依从性（Requirements Compliance）：

数据元素：I_R 表示由于不一致的需求而引起的错误比例；

N_R 表示由于不完整的需求而引起的错误比例；

M_R表示由于曲解的需求而引起的错误比例。

N1表示在一个版本或者模块中不一致的需求数。

N2表示在一个版本或者模块中不完整的需求数。

N3表示在一个版本或者模块中曲解的需求数。

2.1 软件可靠性--度量参数

需求依从性（Requirements Compliance）：

数据元素：

I_R 表示由于不一致的需求而引起的错误比例；

N_R 表示由于不完整的需求而引起的错误比例；

M_R表示由于曲解的需求而引起的错误比例。

N_1表示在一个版本或者模块中不一致的需求数。

N_2表示在一个版本或者模块中不完整的需求数。

N_3表示在一个版本或者模块中曲解的需求数。

计算公式：

$$I_R = (\frac{N_1}{N_1 + N_2 + N_3}) \times 100\%$$

$$N_R = (\frac{N_2}{N_1 + N_2 + N_3}) \times 100\%$$

$$M_R = (\frac{N_3}{N_1 + N_2 + N_3}) \times 100\%$$

$N_1+N_2+N_3$是一个版本或者模块中不一致、不完整、易曲解的需求数之和。

2.1 软件可靠性--度量参数

- 可靠度：

软件可靠度R是指软件在规定的条件下， 规定的时间段内完成预定的功能的概率。或者说是软件在规定时间内无失效发生的概率。

设规定的时间段t_0，软件发生失效的时间为ξ,则

$$R(t_0)=P(\xi>t_0)$$

说明： 该参数是关于软件失效行为的概率描述，是软件可靠性的基本定义，它和硬件可靠性的定义相同。可利用一般的概率规律将它和系统其他部分，如硬件部分， 组合在一起。

该参数适用于对规定时间内无失效工作要求高的系统， 如航空电子系统。

2.1 软件可靠性--度量参数

●失效率：

这两个术语在有关软件可靠性的书籍、文献中经常看到。但它们并不相同，不过有着密切关系。

1）失效率定义

失效率是指在t时刻尚未发生失效的条件下，在t时刻后单位时间内发生失效的概率。即：设ξ为发生失效的时间， Z为失效率，则有

$$Z(t)=\lim_{\Delta t\to 0}\frac{P(t<\xi<t+\Delta t\mid \xi>t)}{\Delta t}=\lim_{\Delta t\to 0}\frac{P(t<\xi<t+\Delta t)}{P(\xi>t)\cdot\Delta t}=\lim_{\Delta t\to 0}\frac{R(t)-R(t+\Delta t)}{R(t)\cdot\Delta t}$$

$$=-\frac{1}{R(t)}\cdot\frac{\mathrm{d}R}{\mathrm{d}t}=\frac{1}{R(t)}\cdot f(t)$$

2.1 软件可靠性--度量参数

- 失效强度
- 假设软件在是t时刻发生的失效数是N(t)， 显然N(t)是一个随机数，且随时间t的变化而不同， 即{N(t), t>0}是一个随机过程。
- 设u(t)为随机变量N(t)的均值， 即有 u(t)=E(N(t))（数学期望），则t时刻的失效强度λ(t)定义为：

$$\lambda(t)=\frac{\mathrm{d}u(t)}{\mathrm{d}t}$$

2.1 软件可靠性--度量参数

- 失效率和失效强度
- 说明：从定义中可以看出， 失效率和失效强度是两个不同的概念， 失效率的定义和靠性中瞬时失效率的定义完全一致的， 是基于寿命的观点给出的。 它是一个条件概率密度。而失效强度则是基于随机过程定义的，是失效数均值的变化率。

- 失效率/失效强度适用于要求失效发生频率比较低的系统， 如操作系统。

2.1 软件可靠性--度量参数

- 一般的软件可靠性参数选取原则

（1）对失效发生频率要求较低的系统，可靠性参数可选失效率或失效强度，如操作系统、电话交换系统软件等。

（2）对在规定时间内能无失效工作要求比较高的系统，可选可靠度作为软件可靠性参数，如火力控制系统软件等。

（3）对使用比较稳定的软件，可选平均失效前时间/平均失效间隔时间作为软件可靠性参数，如通用软件包等。

2.2 软件可靠性分析

- 软件可靠性分析是在软件设计过程中，对可能的失效进行分析，采取必要的措施避免引起失效的缺陷引入软件。
- 软件可靠性分析也可以在系统测试、设计定型或投入使用后， 为纠正措施的制定提供依据，同时为避免类似问题的发生提供借鉴。这些工作将会大大提高使用中软件的可靠性，减少由于软件失效带来的各种损失，提高软件质量。

作业

- 编程实现（基于MTBF的软件系统可靠度计算）
- 已知一个系统在1000小时内出现故障4次，分别计算系统在10小时、100小时、500小时内系统的可靠度（0.96,0.67,0.02）
- 计算在120小时内，可靠度分别为0.9,0.99,0.999的系统出现故障1次的时间（1200小时， 12000小时，120000小时）

第二章 软件可靠性分析

2.1 软件可靠性

2.2 软件可靠性分析

2.3 软件可靠性评估

2.2 软件可靠性分析

共有四种软件可靠性分析方法：

1、软件失效模式和影响分析（Software Failure Modes and Effects Analysis, 软件FMEA）、（只介绍这种）

2、软件故障数分析（Software Fault Tree Analysis, 软件FTA）、

3、将二者结合的软件FEMA与FTA综合分析，以及

4、事件树分析

2.2 软件可靠性分析--软件FMEA

软件失效模式和影响分析（Software Failure Modes and Effects Analysis, 软件FMEA):

- 通过识别软件失效模式，分析造成的后果，研究分析各种失效模式产生的原因，寻找消除和减少其有害后果的方法，以尽早发现潜在的问题，并采取相应的措施，从而提高软件的可靠性和安全性。
- FMEA是一种传统的可靠性、安全性分析方法，在硬件的可靠性工作中已获得了广泛应用，对提高硬件的可靠性、安全性发挥了重要作用。

2.2 软件可靠性分析--软件FMEA

软件FMEA概念的提出始于1979年（Reifer),近年来，软件FMEA的应用有逐步增多的趋势，主要集中在嵌入式软件，并成功应用于安全关键领域，如医疗仪器、军用产品、汽车业等。

Table I
MAJOR ERROR CATEGORIES FOR THREE LARGE SOFTWARE PROJECTS*

MAJOR ERROR CATEGORY	PROJECT I	PROJECT II	PROJECT III		
			OPERATING SYSTEM	APPLICATION SOFTWARE	SIMULATOR SOFTWARE
Computational	9.0	1.7	2.5	13.5	19.6
Logic	26.0	34.5	34.6	17.1	20.9
Data I/0	16.4	8.9	8.6	7.3	9.3
Data Handling	18.2	27.2	21.0	10.9	8.4
Interface	17.0	22.5	7.4	9.8	6.7
Data Definition	0.8	3.0	7.4	7.3	13.8
Data Base	4.1	2.2	4.9	24.7	16.4
Other	8.5	0.0	13.6	9.4	4.9
Total Problem Reports	2019	405	81	275	225

*From [10; Table 17.6] with copyright permission of the authors/publishers.
Body of the Table give the percent of errors in each category for that project.

Reifer J. Software Failure Modes and Effects Analysis[c],IEEE Transactions on Reliability, Vol28 No. 3, Aug 1979: 247-249.

2.2 软件可靠性分析--软件FMEA

软件失效模式（Software Failure Modes）:指软件失效发生的不同方式。例如：输出结果错误、精度不满足要求。

软件失效就是泛指程序在运行中丧失了全部或部分功能、出现偏离预期的正常状态的事件或行为结果。比如：死机、计算结果错误等。

软件失效是有软件故障引起的，归根结底是由软件中潜藏的软件缺陷引起的，如逻辑遗漏、数据错误等。

软件失效模式分析是软件FMEA的基础，只有将被分析对象的所有可能的失效模式尽可能地全面分析出来，才能采取相应措施改进设计，防止失效现象的发生。

2.2 软件可靠性分析--软件FMEA

系统级软件失效模式：国军标软件失效模式指南。

在国军标《GJB/Z 1391-2006故障模式、影响及危害分析指南》中，给出了嵌入式软件FMEA的分析方法，其中软件的失效模式（系统级）

软件失效模式分为两大类：通用失效模式和详细失效模式。

详细失效模式是对通用失效模式的细化，又分为五子类：输入失效、输出失效、程序失效、未满足功能及性能要求失效、其他类型。

在进行系统级软件FMEA时， 应根据被分析软件的不同特点选择合适的失效模式。同时，也可以在分析事假中，不断积累、丰富和完善软件实现模式。

2.2 软件可靠性分析--软件FMEA

系统级软件失效模式：适用于系统级软件FMEA的分析对象。

在IEEE “Standard to Classification for Software Anomalies 93”标准中给出了软件异常的分类。如下页所示：9种系统级软件失效模式，可作为失效模式的参考。

IEEE的软件异常分类方法具有较高的公信度，但该标准不是专门针对软件FMEA制定的， 因此在应用中需根据具体工程项目适当变通。

2.2 软件可靠性分析--软件FMEA

9种系统级软件失效模式：IEEE 软件失效模式参考。

（1） 操作系统挂起

（2） 程序挂起

（3）程序失败：程序不能启动；程序运行不能终止；程序不能退出。

（4）输入问题：错误输入被接受；正确输入被拒绝；描述不正确或遗漏；参数不正确或遗漏。

（5）输出问题： 错误的格式；不正确的结果或数据；不完全或遗漏；拼写问题、语法问题。

2.2 软件可靠性分析--软件FMEA

9种系统级软件失效模式：IEEE 软件失效模式参考。

（6）未达到要求的性能：错误的格式；不正确的结果或数据；不完全或遗漏；拼写问题、语法问题。

（7）发现整个产品失败。

（8）系统错误信息。

（9）其他：程序运行改变了系统配置参数；程序运行改变了其他程序的数据；其他。

2.2 软件可靠性分析--软件FMEA

序号	类别	软件失效模式示例			
1	软件通用失效模式	1） 运行时不符合要求			
		2）输入不符合要求			
		3）输出不符合要求			
2	软件详细失效模式	输入失效	1）未收到输入	输出失效	1）输出结果错误（如输出项缺损或多余等）
			2）收到错误输入		2）输出数据精度轻微超差
			3）收到数据轻微超差		3）输出数据精度中度超差
			4）收到数据中度超差		4）输出数据精度严重超差
			5）收到数据严重超差		5）输出参数不完全或遗漏
			6）收到参数不完全或遗漏		6）输出格式错误
			7）其他		7）输出打印字符不符合要求
					8）输出拼写错误/语法错误
					9）其他

TrIG 华东师范大学软件学院可信智能团队 Trustworthy Intelligence Group

2.2 软件可靠性分析--软件FMEA

序号	类别	软件失效模式示例			
2	软件详细失效模式	程序失效	1）程序无法启动	未能满足功能及性能要求失效	1）未达到功能/性能要求
			2）程序运行中非正常中断		2）不能满足用户对运行时间的要求
			3）程序运行不能终止		3）不能满足用户对数据处理量的要求
			4）程序不能退出		4）多用户系统不能满足用户数的要求
			5）程序运行陷入死循环		5）其他
			6）程序运行对其他单位或环境产生有害影响		
			7）程序运行轻微超时		
			8）程序运行明显超时		
			9）程序运行严重超时		
			10）其他		

2.2 软件可靠性分析--软件FMEA

序号	类别	软件失效模式示例		
2	软件详细失效模式	其他	1）程序运行改变了系统配置要求程序无法启动	6）人为操作错误
			2）程序运行改变了其他程序的数据	7）接口失效
			3）操作系统错误	8）I/O定时不准确导致数据丢失
			4）硬件错误	9）维护不合理/错误
			5）整个系统错误	10）其他

第2.1节作业



华东师范大学软件学院可信智能团队

Trustworthy Intelligence Group

第二章 软件可靠性分析

2.1 软件可靠性

2.2 软件可靠性分析

2.3 软件可靠性评估

TrIG
华东师范大学软件学院可信智能团队
Trustworthy Intelligence Group

2.3 软件可靠性评估

国家标准GB11457中， 软件可靠性评估（Software Reliability Assessment）或软件可靠性评价（Software Reliability Evaluation）是指“确定现有系统或系统部件可靠性所达到的水平的过程”。

国际标准IEEE Std 1633中， 软件可靠性评估被定义为“统计学技术在系统测试和运行期间收集的可观察失效数据上的应用，用于评价软件的可靠性”。

不难看出， 二者的定义均认为，软件可靠性评估是在获得了软件失效数据之后对软件可靠性水平的定量估计和评价。

2.3 软件可靠性评估

软件失效数据可以在下述两种情况下获得：

一是在测试阶段后期，通过软件可靠性测试（即按照软件的实际使用方式测试软件的一种方法），收集测试过程中的失效数据，对软件的可靠性水平进行估计，并能够对未来可能达到的可靠性水平进行预计；

二是在软件投入使用后，通过收集实际使用过程中软件的失效数据，对软件可靠性进行评估，并对未来软件可能达到的可靠性水平进行预计。

2.3 软件可靠性评估

软件可靠性评估结果，不仅可以给出实际的可靠性水平，也可以为下一代软件或同类型软件的可靠性定量要求的确定提供参考。

软件可靠性评估是软件可靠性工程中重要活动内容之一，具有及其重要的作用。

2.3 软件可靠性评估

软件可靠性评估，提及软件可靠性评估模型，它是软件可靠性定量评估技术的基础。

软件失效具有随机性。软件在是t时刻发生的失效数是N(t)， 显然N(t)是一个随机数，且随时间t的变化而不同， 即{N(t), t>0}是一个随机过程。

问： 这个随机过程应该服从什么分布？

2.3 软件可靠性评估

问： 这个随机过程应该服从什么分布?

参照国际标准IEEE Std.1633，可将软件可靠性评估模型分为3中通用类别:

- 指数分布的非齐次泊松(Poisson)过程模型(Non-homogeneous Poisson process，NHPP)、
- 非指数分布的NHPP模型和贝叶斯（Bayesian）模型。

2.3 软件可靠性评估

呈指数分布的NHPP模型

- 呈指数分布的NHPP模型使用随机过程和失效率函数方法。
- 失效率函数Z(t)是运行时间t的函数。
- 具有代表性的模型是Jelinski-Moranda模型， 1972年开发的可靠性模型，是最早建立的软件可靠性模型之一，曾用于麦克唐奈道拉斯海军工程中。现在大多数软件可靠性模型要么可认为是其变形或扩展，要么与其密切相关。
- ◆ 该模型是软件可靠性研究领域分第一个里程碑。

2.3 软件可靠性评估

Jelinski-Moranda模型

（1） 假设与数据要求：

模型的基本假设如下：

① 程序中的固有错误数N_0是一个未知的常数；

② 程序中的各个错误是相互独立的，每个错误导致系统发生失效的可能性大致相同，各次失效间隔也是相互独立的；

③ 测试过程中检测到的错误都被排除，每次排错只排除一个错误，排错时间可以忽略不计，在排错过程中不引入新的错误；

2.3 软件可靠性评估

Jelinski-Moranda模型

（1） 假设与数据要求：

模型的基本假设如下：

④ 程序失效率在每个失效间隔时间内是常数，其数值正比于程序中残留的错误数，在第i个测试区间，其失效率函数为

$$Z(x_i)=k(N_0-i+1)$$

式中：k为比例常数， x_i为第i次失效间隔中以i-1次失效为起点的时间变量。

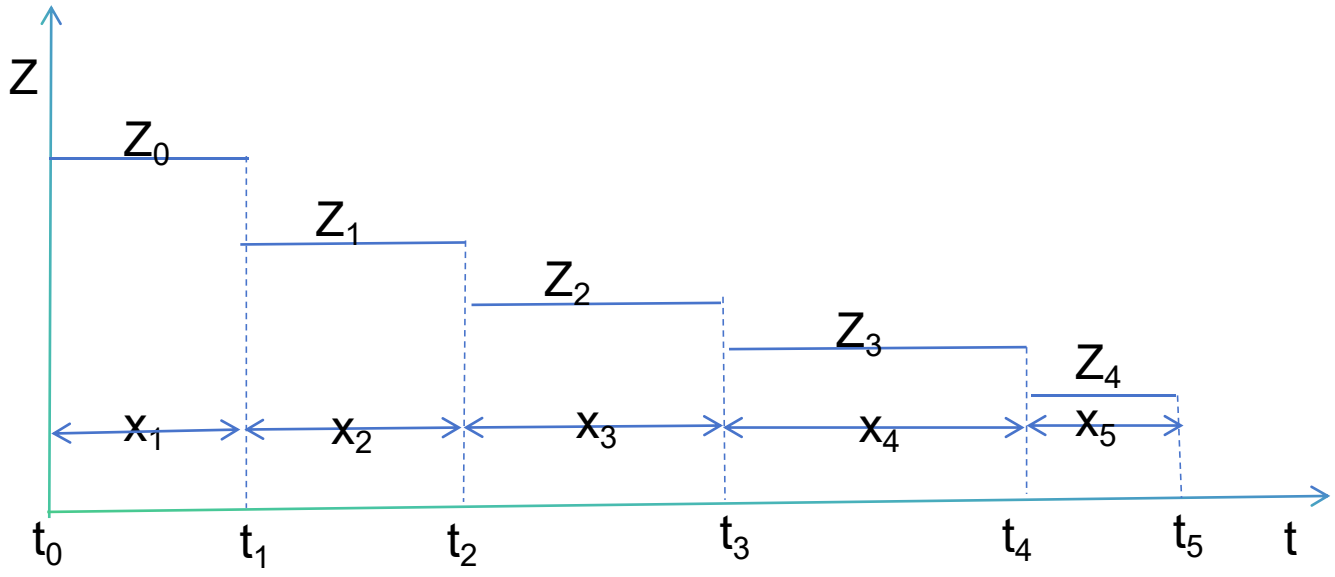
2.3 软件可靠性评估

Jelinski-Moranda模型

在第i个测试区间，其失效率函数为

$$Z(x_i)=k(N_0-i+1)$$

式中：k为比例常数， x_i为第i次失效间隔中以i-1次失效为起点的时间变量。



图是J-M模型失效率随时间变化的曲线。图中ti是以时间0点为起点的第i次失效的累计发生时间， 即$x_i=t_i-t_{i-1}$，且$i\geq1$，$t_0=0$。

2.3 软件可靠性评估

Jelinski-Moranda模型

（1） 假设与数据要求：

模型的基本假设如下：

⑤ 错误以相等的可能发生，且相互独立，错误检测率正比于当前程序中的错误数。

⑥ 软件的运行方式与预期的运用方式相似。

2.3 软件可靠性评估

Jelinski-Moranda模型

（2） 模型构造与参数估计：

在假设的基础上，运用可靠性工程学的基本理论，以第i-1次失效为起点的第i次失效发生的时间是一个随机变量，它服从以K（N_0-i+1）为参数的指数分布，其（失效）密度函数为

$$f(x_i)=\mathrm{k}(N_0-i+1)e^{-\mathrm{k}(N_0-i+1)x_i}$$

2.3 软件可靠性评估

Jelinski-Moranda模型

（2） 模型构造与参数估计：

它服从以k(N_0-i+1)为参数的指数分布，其（失效）密度函数为

$$f(x_i)=\mathrm{k}(N_0-i+1)e^{-\mathrm{k}(N_0-i+1)x_i}$$

其(失效)分布函数

$$F(x_i)=\int_0^{x_i} f(x)\mathrm{d}x=1-e^{-\mathrm{k}(N_0-i+1)x_i}$$

2.3 软件可靠性评估

Jelinski-Moranda模型

（2） 模型构造与参数估计：

其（失效）分布函数

$$F(x_i)=\int_0^{x_i} f(x_i)\mathrm{d}x_i = 1-e^{-\mathrm{k}(N_0-i+1)x_i}$$

其可靠性函数为

$$R(x_i)=1-F(x_i)=e^{-\mathrm{k}(N_0-i+1)x_i}$$

系数k和程序故有错误数N_0确定后，可靠性函数就确定了。

2.3 软件可靠性评估

Jelinski-Moranda模型 系数k和程序故有错误数N_0确定后，可靠性函数就确定了。

（2） 模型构造与参数估计：

假设总共发生了n个失效，则似然函数为

$$L(x_1,\cdots,x_n)=\prod_{i=1}^{n}f(x_i)=\prod_{i=1}^{n}\mathrm{k}(N_0-i+1)e^{-\mathrm{k}(N_0-i+1)x_i}$$

对上式两边取对数，得

$$\ln L(x_1,\cdots,x_n)=\sum_{i=1}^{n}\ln f(x_i)=\sum_{i=1}^{n}(\ln \mathrm{k}(N_0-i+1)-\mathrm{k}(N_0-i+1)x_i)$$

2.3 软件可靠性评估

Jelinski-Moranda模型 系数k和程序故有错误数N_0确定后，可靠性函数就确定了。

（2） 模型构造与参数估计：

假设总共发生了n个失效，则似然函数为 $L(x_1,\cdots,x_n)=\prod_{i=1}^{n} f(x_i)=\prod_{i=1}^{n} \mathrm{k}(N_0-i+1)e^{-\mathrm{k}(N_0-i+1)x_i}$

则模型参数的极大似然法估计值是以下方程组的解：

$$\begin{cases} \hat{\mathrm{k}}=\dfrac{\mathrm{n}}{\hat{N}\left(\sum_{i=1}^{n} x_i\right)-\sum_{i=1}^{n}(i-1)x_i} \\ \sum_{i=1}^{n}\dfrac{1}{\hat{N}-(i-1)}=\dfrac{n}{\hat{N}-(1/\sum_{i=1}^{n} x_i)(\sum_{i=1}^{n}(i-1)x_i)} \end{cases}$$

其中 $\hat{\mathrm{k}}$ 和 $\hat{N}$是模型参数k和N_0的点估计值。
这是一个超越方程，可用数值计算法求解方程组，可以得到模型参数k和N_0的点估计值。

2.3 软件可靠性评估

Jelinski-Moranda模型 系数k和程序固有错误数N_0确定后，可靠性函数就确定了。

（3） 可靠性预计：

运用可靠性工程学的基本理论，利用上述推导出的估计值 $\hat{k}$ 和 $\hat{N}$，可相应地求得以下可靠性参数的估计值：

① 可靠度：

$$R_{n+1}(x) = R(x \mid t_n) = e^{-(\hat{N}-n)\hat{k}x}$$

② 不可靠度：

$$F_{n+1}(x) = 1 - e^{-\hat{k}(\hat{N}-n)x}$$

2.3 软件可靠性评估

Jelinski-Moranda模型 系数k和程序固有错误数N_0确定后，可靠性函数就确定了。

（3） 可靠性预计：

运用可靠性工程学的基本理论，利用上述推导出的估计值 $\hat{k}$ 和 $\hat{N}$，可相应地求得以下可靠性参数的估计值：

③失效密度：

$$f_{n+1}(x) = -\frac{dR_{n+1}(x)}{dx} = (\hat{N}-n)\hat{k}e^{-(\hat{N}-n)\hat{k}x}$$

2.3 软件可靠性评估

Jelinski-Moranda模型 系数k和程序固有错误数N_0确定后，可靠性函数就确定了。

（3） 可靠性预计：

运用可靠性工程学的基本理论，利用上述推导出的估计值 $\hat{k}$ 和 $\hat{N}$，可相应地求得以下可靠性参数的估计值：

④ 平均失效前时间MTTF：给定第n个软件失效发生时刻t_n，由失效独立性假设知，t_n时刻之后软件失效的MTTF为：

$$T_{TF_{n+1}} = E\{X_{n+1} \mid x_1, \cdots, x_n\} = \int_0^{\infty} R_{n+1}(x)\mathrm{d}x = \frac{1}{k(N_0 - n)}$$

则 $$\hat{T}_{TF_{n+1}} = \frac{1}{\hat{k}(\hat{N} - n)}$$

由此我们可以估计出软件可靠性的参数。他们都是依赖于前n次失效时刻，依据这n次失效时间，估计出了J-M模型的参数k和N0。n值越大估计的准确性就越高。

第二章作业

1）总结软件可靠性定义以及相关度量参数，字数不少于8000字

2）总结系统级软件FMEA的定义，并阐述9种系统级软件失效模式。

3）假定某一软件在测试过程中，已发现了10次失效，其时间分别为

失效编号	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
失效时间	15	13	9	10	21	23	18	22	17	16

基于J-M模型，估计第11次失效发生时，软件可靠度、不可靠度、失效密度和失效前平均时间MTTF的值。